CHAPTER 11

Design A Notification System

Deliver push, SMS and email notifications reliably at millions per day.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a scalable notification system that sends **mobile push (iOS/Android), SMS, and email** (and possibly in-app) notifications triggered by services (e.g. shipping updates, marketing campaigns, OTP). Notifications can be real time or scheduled.

Why interviewers ask it: Illustrates decoupling with message queues, third-party integrations, retries, de-duplication, templates and user preferences.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Types supported?	Push, SMS, email.
Real-time?	Soft real-time; slight delay under load is acceptable.
Devices?	iOS, Android, laptop/desktop.
Who triggers?	Client apps and server-side scheduled jobs.
Opt-out?	Yes, users can opt out.
Volume?	10 M mobile push, 1 M SMS, 5 M email per day.

3. Requirements

Functional

- Send notifications over multiple channels.
- Respect user settings (opt-out, per-channel).
- Support templates, scheduling, and batch/campaign sends.
- Track delivery status.

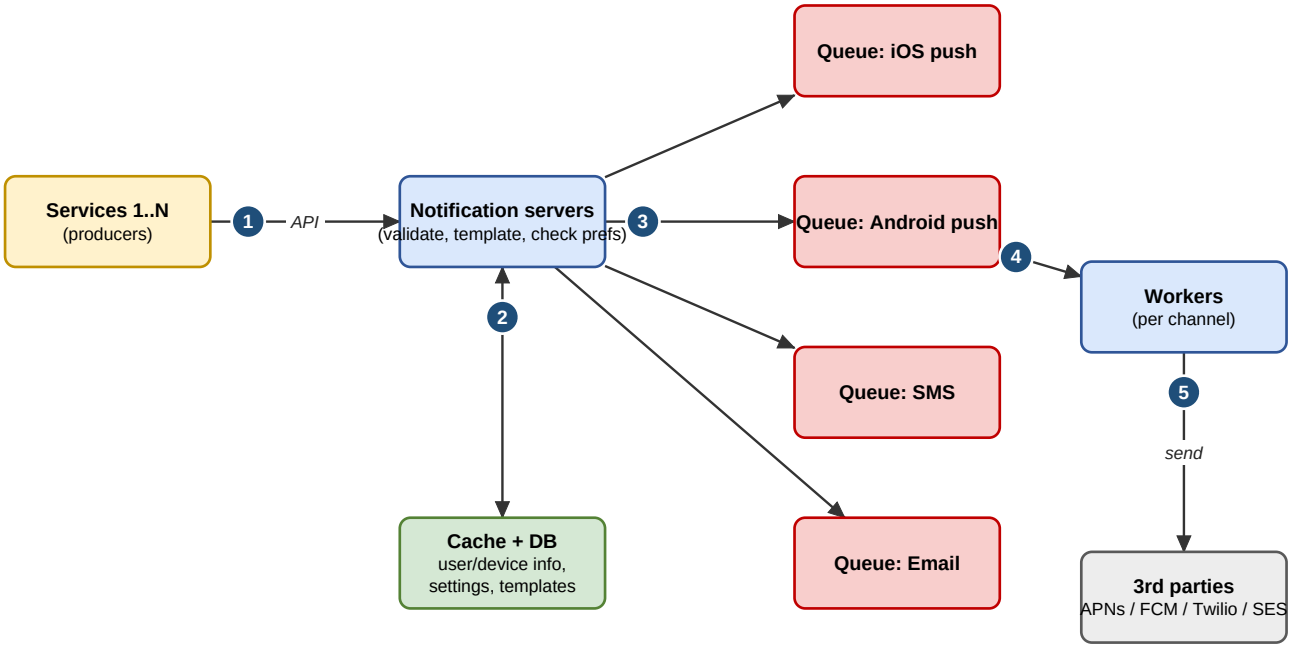
Non-functional

- Reliable: **no data loss** (delay is ok; losing is not).
- Soft real-time; handle spikes.
- Highly scalable; extensible to new channels/vendors.
- Avoid duplicates (best effort: exactly-once is not achievable end to end, so aim for at-least-once + dedupe).

4. Back-of-the-envelope estimation

- Total 16 M notifications/day -> about 185/s average; peak (campaigns) maybe 10-50x -> 2-10 K/s.
- Notification event about 1 KB -> 16 GB/day logged; keep 30-90 days of history.

5. High-level design



One queue per channel isolates failures: an SMS vendor outage never blocks push or email.

Step	What happens
1	A producer service calls the notification API with the recipient, channel(s), template ID and parameters. The API **returns 202 Accepted** quickly.
2	Notification servers validate the call, load **user and device info, opt-in settings and templates** (cache first), and apply rate limits.
3	The rendered notification is put on the **channel's queue** (here Android push); other channels have their own queues (iOS, SMS, email), so an outage in one never blocks the others.
4	Workers for that channel pull messages at their own pace (natural back-pressure), de-duplicate by event ID, and call the vendor.
5	The worker sends via the third party (FCM, APNs, Twilio, SES). On failure it **retries with exponential backoff**, then sends to a dead-letter queue and alerts.

Request flow

- A service calls the notification API (`POST /v1/notifications` with recipient, channel(s), template id, params).
- Notification servers validate, fetch user/device tokens and opt-in settings (cache first, DB on miss), render the template, and enqueue to the channel queue.
- Channel workers pull messages, call the third-party provider (APNs, FCM, Twilio/Nexmo, SendGrid/SES), and record status.
- On failure workers retry with exponential backoff; after N attempts the message goes to a dead-letter queue and an alert fires.

6. Deep dive

Why a message queue per channel

- Decouples producers from slow/unreliable vendors; absorbs bursts; lets each worker pool scale independently; isolates outages.

Contact info gathering

- At sign-up/app install, store device tokens, phone numbers, emails in a **user / device** table (one user -> many devices).

Reliability: no data loss

- Persist every notification in a **notification log DB** before/while enqueueing; workers retry from the queue (durable, e.g. Kafka/SQS/RabbitMQ).
- At-least-once delivery means possible duplicates.

Deduplication

- Each event has a unique event ID; workers check 'seen before?' (Redis set with TTL) before sending; if seen, drop. Not perfect, but greatly reduces duplicates.

Rate limiting and user settings

- Cap notifications per user per time window to avoid spamming (chapter 05). Check `notification_setting(user_id, channel, opt_in)` before sending.

Templates, scheduling, retries

- Notification templates (reuse format with parameters) keep messages consistent and reduce errors.
- Scheduled sends: store with a send-at time; a scheduler (cron / delayed queue) enqueues when due.
- Retry mechanism with exponential backoff; dead-letter queue for permanent failures.

Security

- Authenticate callers with appKey/appSecret so only verified clients can send; encrypt sensitive payloads; validate recipients.

Monitoring and tracking

- Monitor queue depth (backlog means more workers needed), vendor latency/errors, delivery rate. Track open/click events via analytics service.

Multi-vendor failover

- Wrap vendors behind an adapter; fail over to a secondary provider (e.g. two SMS gateways) on error spikes.

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Delivery guarantee	At-most-once (may lose) / At-least-once + dedupe	At-least-once with idempotent dedupe; exactly-once end-to-end is not achievable with 3rd parties.
Push vs pull	Producer pushes into queues / workers pull	Workers pull for natural back-pressure.
Single queue vs queue per channel	Simple / isolated and independently scalable	Per-channel (and per-priority) queues.
Sync vs async API	Wait for delivery / return 202 Accepted	Async; expose status via ID.

8. Scaling, reliability, failure modes

- Stateless notification servers behind an LB; shard the log DB; cache user settings.
- Priority queues: OTPs and security alerts ahead of marketing.
- Batch push/email to vendors where supported to cut API calls.

9. Likely follow-up questions

- How do you guarantee an OTP SMS is delivered fast even when marketing floods the system?
- How do you avoid sending duplicate notifications on retries?
- How do you handle a vendor outage?
- How would you add WhatsApp or in-app notifications?

10. Common mistakes

- Single point of failure notification server or single shared queue.
- Calling vendors synchronously from the producer's request path.
- No opt-out / rate limit -> spam and compliance issues.

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"I'll support push, SMS and email, soft real time, and both server-triggered and scheduled notifications, with user opt-out. Volume is about 10 M push, 1 M SMS and 5 M email a day, so roughly 185 per second on average and a few thousand at campaign peaks. The key requirement is that we must not lose a notification; a short delay is acceptable."
2. High-level design	10 min	"Producer services call notification servers, which validate, check user settings and templates, and put messages on one queue per channel. Workers pull from the queues and call third-party providers: APNs, FCM, Twilio and an email provider. I use queues so slow or failing vendors never block producers."
3. Deep dive	25 min	"For reliability, every notification is persisted in a log and queues are durable, with at-least-once delivery. To avoid duplicates, each event has an ID and workers check a Redis set before sending; exactly-once is not achievable with third parties. Retries use exponential backoff and a dead-letter queue. I'd add per-user rate limits, priority queues for OTPs, templates, scheduling, and opt-out checks."
4. Wrap up	4 min	"Monitoring: queue depth, delivery rate, vendor errors. For security, authenticate senders with app keys. For vendor outages, adapters with a secondary provider. Adding WhatsApp or in-app is just another queue and worker pool."

12. 60-second recap / practice checklist

- Producers -> notification servers -> per-channel queues -> workers -> third-party providers.
- Reliability via durable queues + notification log + retries + DLQ; dedupe via event ID.
- Settings, templates, rate limit, priority, scheduling, monitoring.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• GeeksforGeeks: Design a notification system
• Apple Push Notification service
• Firebase Cloud Messaging